

Relay Miner capacity

The same load, run on the same machine against **main**, the version operators run today, and against **this release**. The chain settled **1,501 of 1,501** claims with valid proofs, even though the miner was killed on purpose 2 times, once in a proof window and once in a claim window.

This release is PR #44 (3e99948) on smt v0.15.0, which already carries smt#55 (frees from the miner's memory the value of leaves already written to Redis), plus smt#56 (each tree node encoded once). The release ships only once smt#56 is published and pinned.

How it was tested

Everything below this section refers to this setup; it is not repeated.

4 terms used throughout. A session is 1 (application, service) pair served by 50 suppliers, and each supplier keeps 1 Merkle tree (SMST) of it in Redis and submits 1 claim for it. The 6 sessions born in the same block are a *generation*: $6 \times 50 = 300$ trees and 300 claims. A tree is *live* from its first relay until the miner compacts it at the claim. After a session ends, its claim is submitted during the *claim window*, and its proof during the *proof window* that follows. Heights are written h31 for height 31.

SET · 1 OF EACH · CONTAINER LIMITS

Component	main	this release	What it does
relayer	8 GiB	8 GiB	validates ring signatures, charges the stake, signs responses, publishes to Redis
miner	24 GiB	8 GiB	builds the SMST trees, submits claims and proofs
redis	20 GiB · maxmemory 16.8 GiB	16 GiB · maxmemory 12.8 GiB	all session state. There is no local disk anywhere
validator	60 s configured · ~64 s measured	60 s configured · ~60 s measured	the chain (pocketd 0.1.35, smt v0.14.1), reset to genesis before each run

- **Resources, main against this release.** Main ran with more memory so that its real peak could be measured without an OOM kill; its Redis limit is from the run's notes, not from a saved config. The validator was configured the same in both runs, yet main's blocks measured ~7 % longer, before and after its load, for a reason not found; every per-height comparison gives main slightly more time and more relays per height.
- **Load.** ~2,400 relays/s over 3 transports (jsonrpc, websocket, grpc), from h5 to h81. This is the load chosen for the test, not a measured ceiling. It was sent by the project's own CLI straight to the relayer. Main's load was stopped at h59 by the run's safety stop, with Redis near its budget. 6 services, 2 per transport, each driven by 1 application across the 50 suppliers of its session. Every response is verified, so a relay counts as served only if it is valid.
- **1 MiB burst.** At h44 a second client sent 1 MiB bodies to develop-http, 128 at a time, for 15 seconds, on top of the full load.
- **Kills.** `kill -9` sent from the Kubernetes node, which reproduces a kernel OOM kill: once in a proof window (h63) and once in a claim window (h72), on 2 different generations. Main's kill never fired: main marks a session proved when it sends the proof, so the trigger found no work in flight.

One setup. These figures come from one topology on one machine. Supplier count, services, validation mode, transports and hardware all change them. Use them as a reference for sizing your deployment, not as a guarantee.

Against main: steady state

The miner uses 80 % less memory and Redis 86 % fewer operations; the relayer spends 54 % more CPU. Heights 20–42: full load, before the burst. Memory is the highest reading up to h43; CPU, throughput and operations are the mean.

STEADY STATE · HEIGHTS 20–42

What	main	this release	Change
miner memory, peak	14,204 MiB	2,791 MiB	−80 %
miner memory at h40	13,472 MiB	2,207 MiB	−84 %
redis memory, peak	7,346 MiB	4,375 MiB	−40 %
redis CPU	1.43 cores	0.32 cores	−78 %
redis operations per second	46,081	6,331	−86 %
redis operations per relay	19.3	2.6	−86 %
miner CPU	0.94 cores	0.52 cores	−45 %
relayer CPU	2.35 cores	3.63 cores	+54 %
relayer memory, peak	88 MiB	118 MiB	+34 %
relays per second	2,389	2,400	+0 %

Read with 2 caveats

1. The gain belongs to all the commits between main and this release, not to any single one. Each figure comes from 1 run per build.
2. The relayer's +54 % CPU was investigated without a conclusion. 8 causes were ruled out, each with its own measurement: double validation, relay counting, the load client and payload, the ring size, the worker-pool size, idle CPU, the toolchain, and the ring-signature verification itself, benchmarked on 1 core.

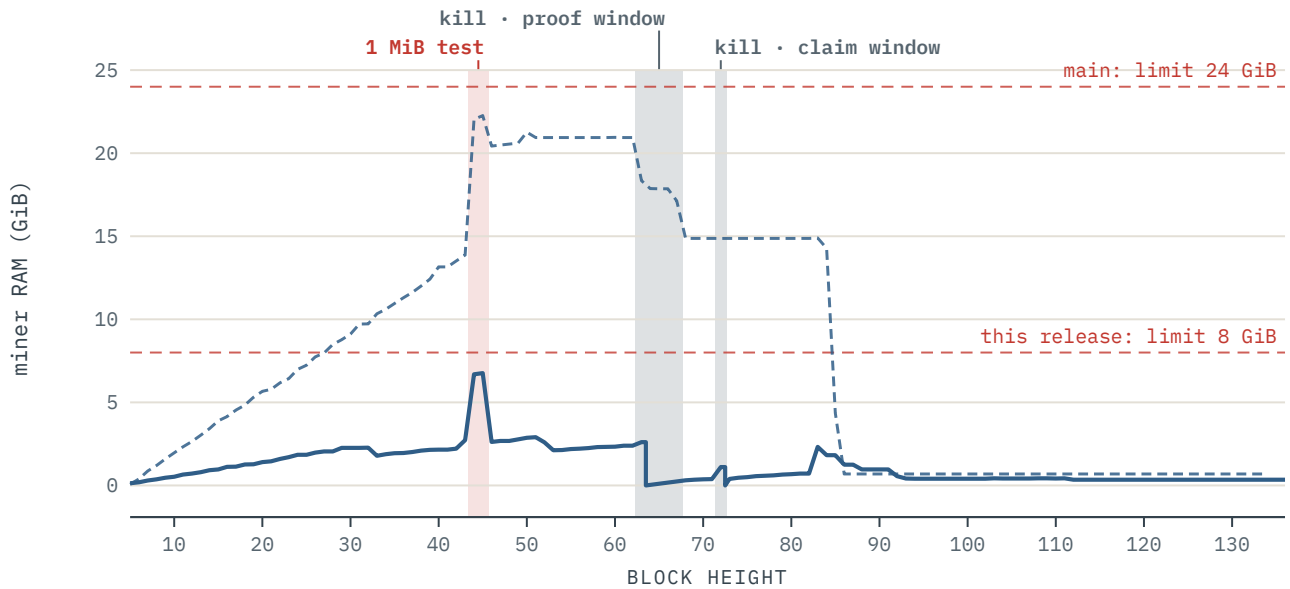
Against main: height by height

The 2 runs on 1 height axis. Dashed is main, solid is this release. The red band is the 1 MiB burst; grey marks this release's 2 kills.

CHART 1 · MAIN VS THIS RELEASE

Miner memory

--- main — this release — miner

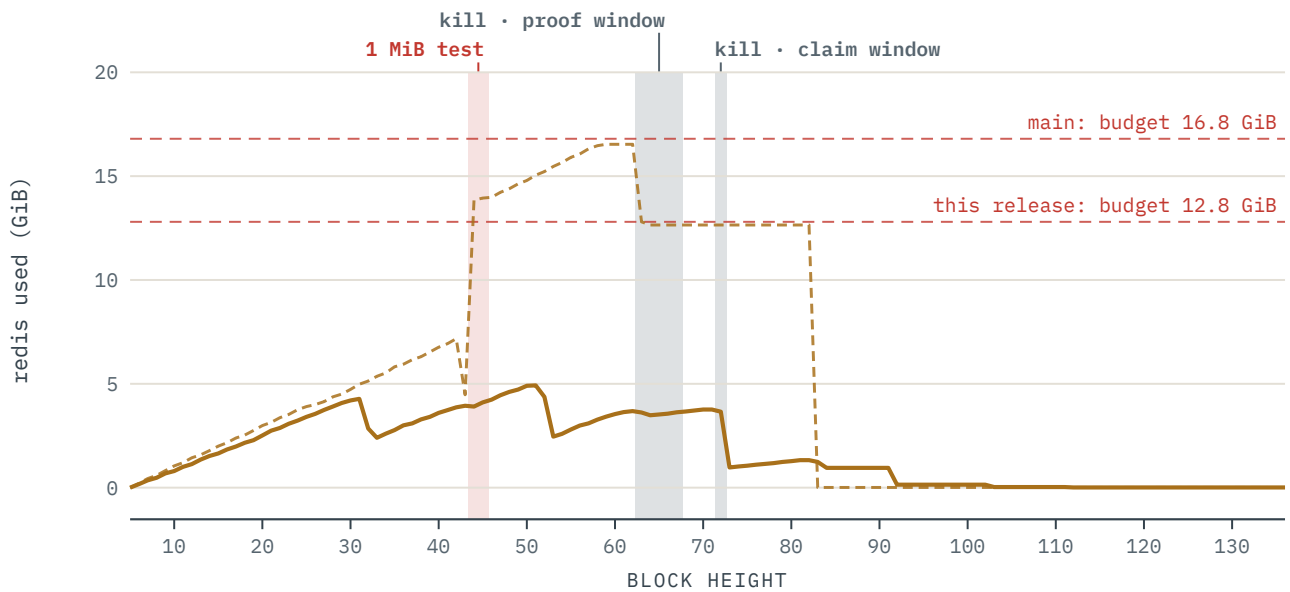


Under load main's miner grows to 13.9 GiB by h43 and 22.3 GiB after the burst, and frees it only at its proofs. This release stays under 3 GiB, rises to 6.8 GiB in the burst and is back under 3 GiB within 2 blocks. At each kill its memory falls to 0 and the new process starts again; after the first kill the metric kept reporting the dead process for 4 blocks, so the rise to h68 is drawn, not measured.

CHART 2 · MAIN VS THIS RELEASE

Redis memory

--- main — this release — redis

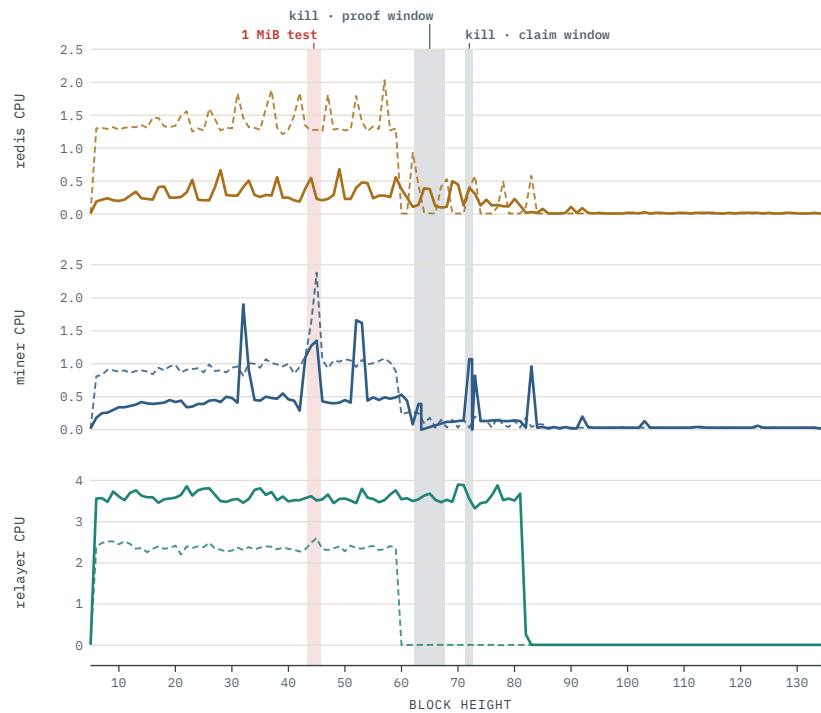


This release frees Redis when a generation is claimed (h33, h53, h73); main only when it sends the proof, 11 blocks later. Main's step at h44 is the burst.

CHART 3 · MAIN VS THIS RELEASE

CPU by component

--- main — this release — redis — miner — relayer

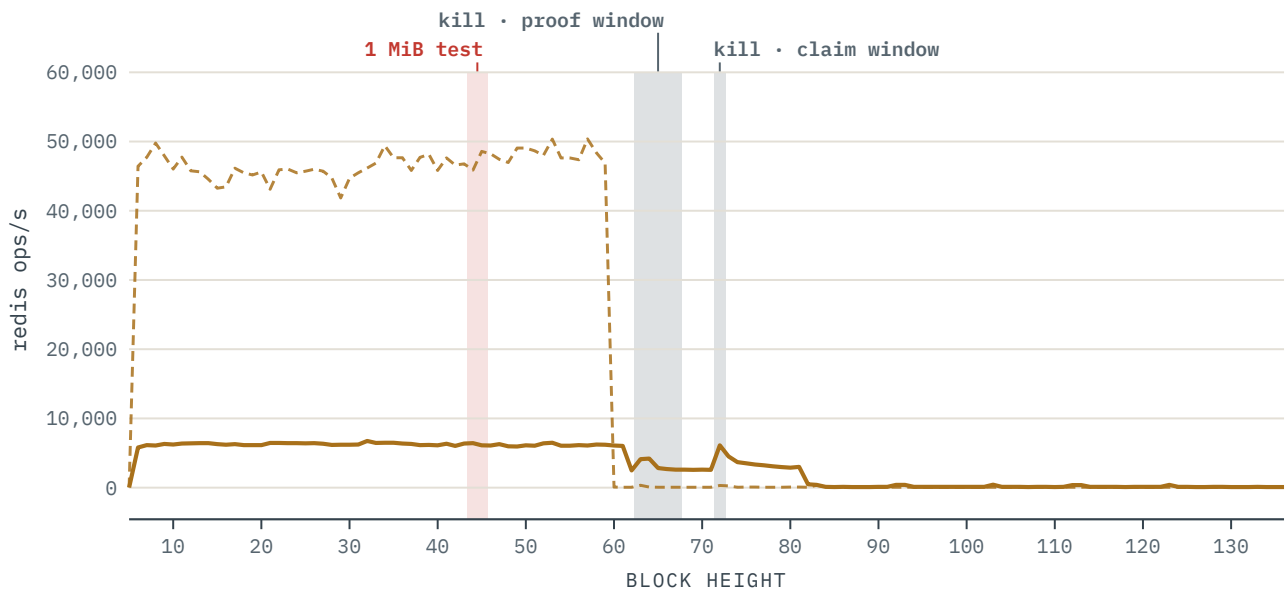


The miner's spikes are claim and proof builds. The relay is the one line above main.

CHART 4 · MAIN VS THIS RELEASE

Redis operations per second

--- main — this release — redis ops/s

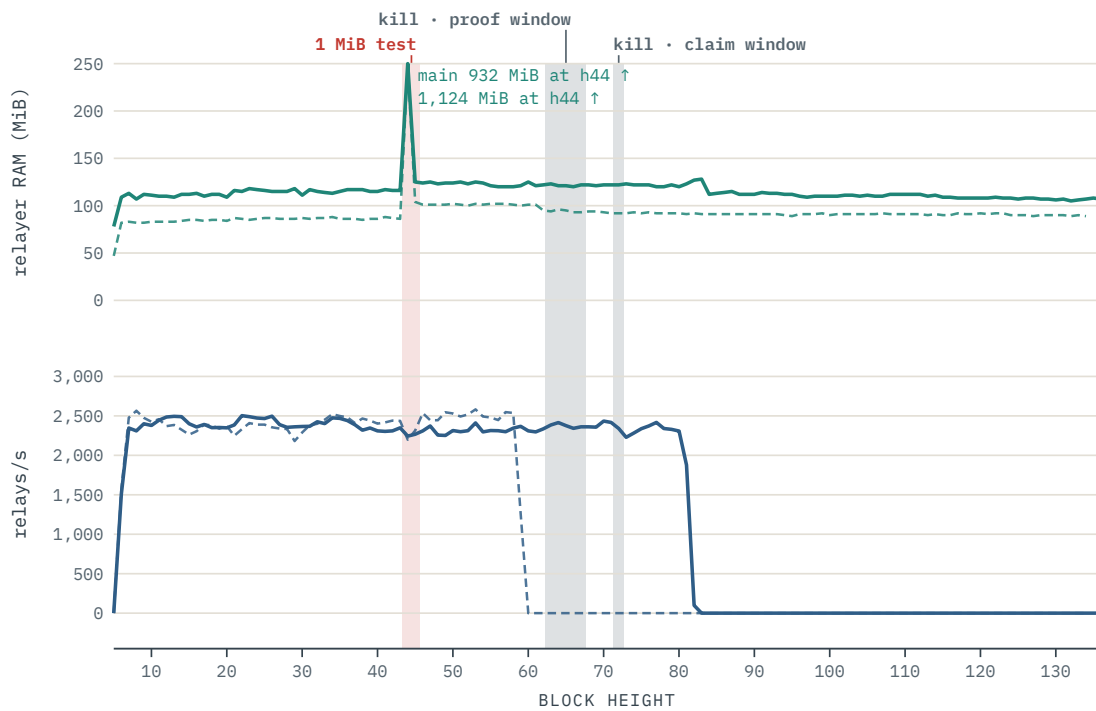


This release's operations halve from h62, when the higher difficulty lets fewer relays into the trees; the gap to main holds throughout.

CHART 5 · MAIN VS THIS RELEASE

Relayer memory and throughput

--- main — this release — relayer RAM — relays/s



Mean throughput over h20–42 is within 1 %. The relayer's spike at h44 is the 1 MiB bodies it accepted and held while publishing them.

Against main: the 1 MiB burst

This release refused nothing and survived; main accepted every body and grew Redis by ~10 GiB.

1 MIB BURST · H44

What	main	this release
relays refused, publish queue full	0	0
throughput, lowest height of the burst vs h43	2,199 vs 2,437	2,244 vs 2,349
redis memory, h43 → h47	4,582 → 14,563 MiB	4,037 → 4,551 MiB
relayer memory, peak	932 MiB	1,124 MiB
miner memory, peak	22,792 MiB	6,921 MiB

Main has no ceiling on its publish queue (the buffer between the relayer and Redis): it accepted every body, and Redis grew by ~10 GiB within 4 blocks and held it until the next proof. This release bounds each stage of that path in bytes, per service, and when it is full it closes WebSocket connections with code 1013 (try again later). It accepted ~6,700 of the 1 MiB bodies (~7 GB before compression) and refused nothing on any service. The other transports still slowed while the burst lasted: for about a minute grpc fell from 797 to 650 relays/s and websocket from 809 to 701, then both recovered within 25 s.

The Redis row flatters this release. The bodies are generated filler, and relays of 16 KiB or more travel compressed, so they left no step in Redis above its normal growth. A real LLM prompt compresses far less.

Why the release requires smt#56. Without it, this burst killed the miner for running out of memory at its 8 GiB limit. The cause was garbage, not retained memory: the SMST library encoded every node 2 times each time the tree was written and copied large leaves, ~4 leaf-sized allocations per relay, 2.6 GB/s during the burst. smt#56 fixes it with roots and proofs byte-identical to the unmodified library's, checked by a test against ones recorded from it. With it the allocation rate peaks at 0.85 GB/s and the miner survives, at 6.8 of 8 GiB.

This release: height by height

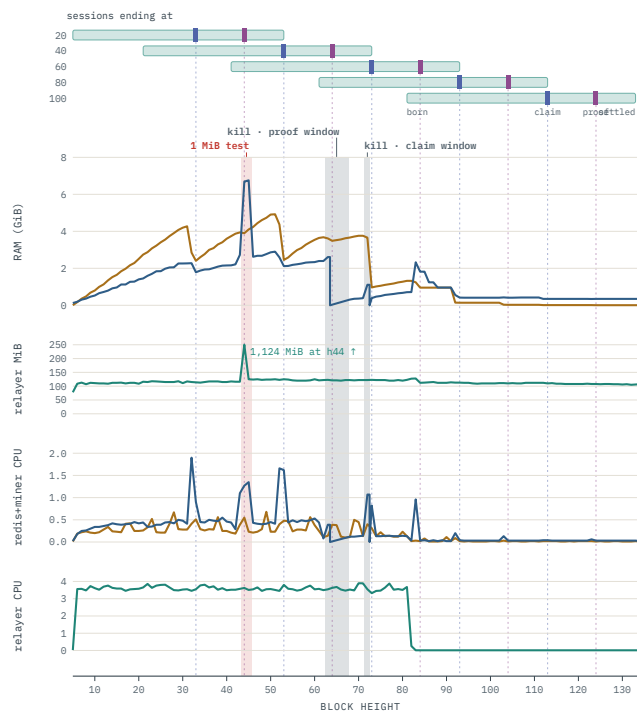
This release alone, with the sessions drawn on the axis: the 5 lanes at the top are the 5 generations, each running

from birth to settlement with its claim and proof marked.

CHART 6 · THIS RELEASE

Resources, with the generations on the axis

redis miner relayer generation claim tx proof tx



RAM is the container working set; Redis is `used_memory` as Redis reports it. The relayer has its own panels because on a shared axis it would read as zero.

CHART 7 · THIS RELEASE

The load, split by transport

jsonrpc websocket grpc



780, 788 and 794 relays/s on average over h20–60 for jsonrpc, websocket and grpc: the load was split evenly and the relayer does not favour a transport. These are the rates sent, not ceilings: all 3 shared 1 relayer and the same backend. The kills leave no mark; the burst is the dip at h44.

Sizing this release

Size the relayer by CPU, and the miner and Redis by the sessions they hold. The miner keeps the tree of every live session in memory, so its RAM follows the same overlap of generations as Redis (≤ 2.9 GiB at 300–600 live trees), and a burst of large requests adds on top of it (6.8 GiB). The rates below are what 1 relayer served under the test load, not its ceiling; the relayer keeps no state and scales horizontally, so more throughput means more replicas. The peak is set by the protocol's windows.

WHAT TO EXPECT · 10,743,008 RELAYS SERVED · HIGHEST READING OUTSIDE THE BURST UNLESS STATED

Component	Sustained	What it costs	Read it as
relayer	~2,400 relays/s	≤ 3.9 cores · ≤ 128 MiB	the test load, every response signed; not a ceiling. Its memory is negligible
miner	300–600 live trees	0.5 cores steady · 1.9 at a claim · ≤ 2.9 GiB	CPU comes in bursts, once per generation, when ~300 claims are built
miner, 1 MiB burst	128×1 MiB for 15 s	6.8 GiB of 8 GiB	survives, but this is the thinnest margin in the run
redis	4.9 GiB peak	≤ 0.7 cores	38 % of the 12.8 GiB budget. Memory is released at every claim

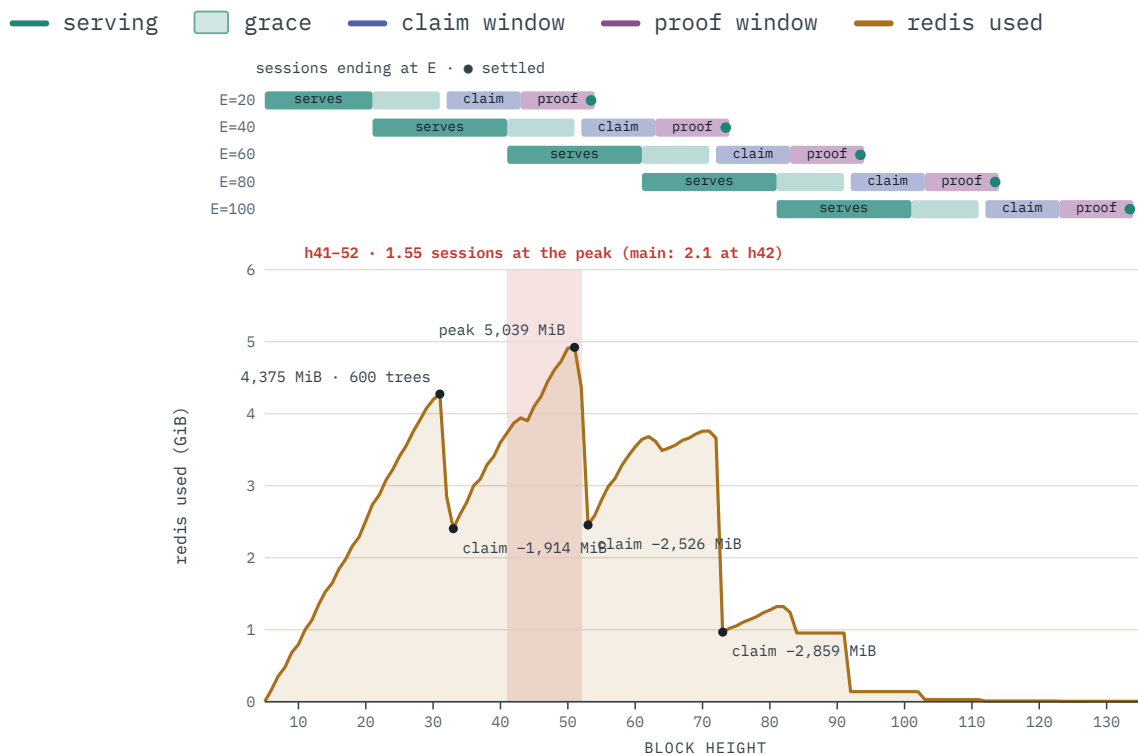
Why the peak has this shape

The chain's windows allow up to 2.6 sessions' worth of relays to exist at once, and no optimization moves that ceiling. The offsets come from `pocketd query shared params` on this run's chain (re-queried after the run). For a session ending at height E:

num_blocks_per_session	20	session	= E-19 ... E	
grace_period_end_offset_blocks	10	grace	= E+1 ... E+10	← relays still arrive
claim_window_open_offset_blocks	11	claim opens	= E+11+1	
claim_window_close_offset_blocks	10	claim closes	= opens + 10	
proof_window_open_offset_blocks	1	proof opens	= closes + 1	
proof_window_close_offset_blocks	10	proof closes	= opens + 10	→ settlement
E=20	serves 1...20	claim 32...42	proof 43...53	settled 53
E=40	serves 21...40	claim 52...62	proof 63...73	settled 73
E=60	serves 41...60	claim 72...82	proof 83...93	settled 93

CHART 8 · THIS RELEASE

Each generation's windows over Redis memory



The lanes are computed from the shared params; the curve is measured. Claims landed 1 block after each window opened (h33, h53, h73), and each is a drop in the curve.

The count. The ceiling is reached when a tree waits for the end of its proof window (E+33) while the next generation is complete and the one after has served 12 blocks: $1.0 + 1.0 + 0.6 = 2.6$ sessions. What each build reaches depends on when it lets a tree go:

- **Main: 2.1 sessions at h42.** It keeps a tree until its proof and sends the proof early in the window (h44 for E=20). At h42, E=20 (1.0) and E=40 (1.0) are complete and E=60 has served 2 blocks (0.1): 900 live trees, 7,346 MiB.
- **This release: 1.55 sessions at h51.** The miner compacts a tree at its claim, so E=20 left the trees at h33. At h51, E=40 is complete (1.0; its claim window opens only at E+12 = 52, and its relays arrive until E+10 = 50) and E=60 has served 11 blocks (0.55): 601 live trees, 5,039 MiB.

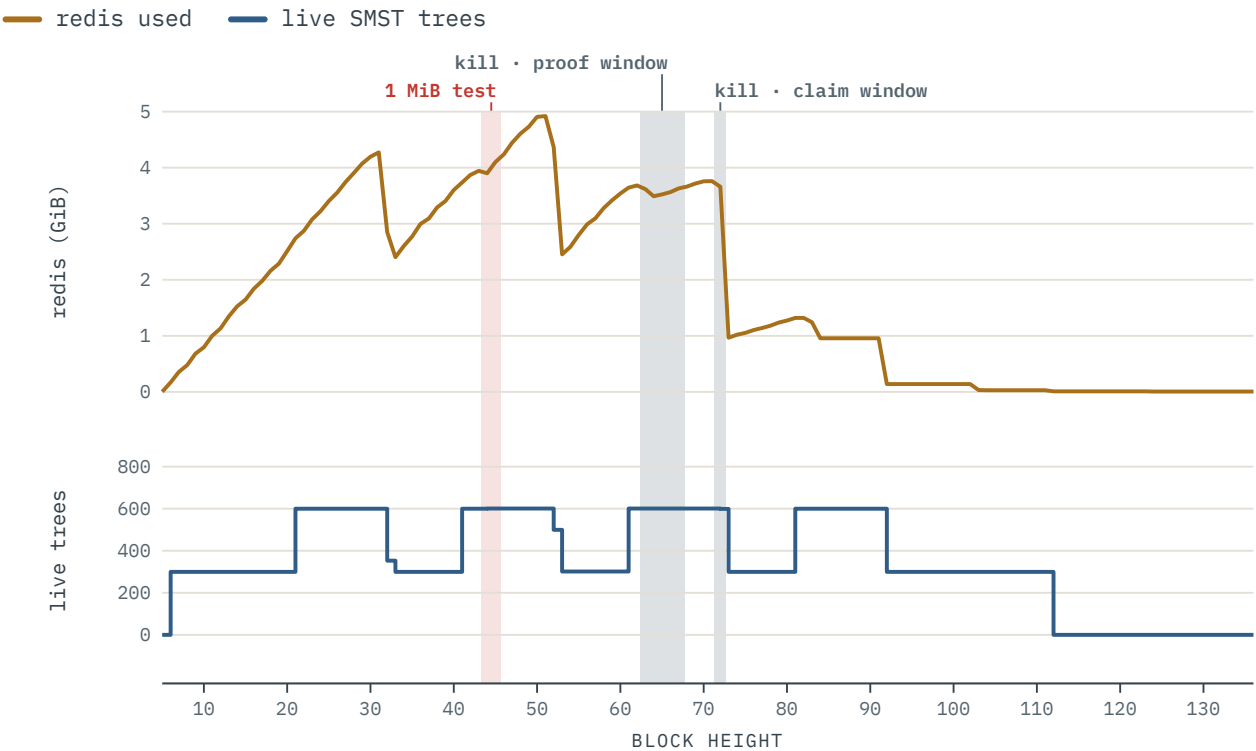
That overlap repeats every 20 blocks and sets the peak however efficient each tree is: the 5,039 MiB at h51 is that overlap, not a leak. A generation takes 33 blocks from the end of its session to settlement, about 33 minutes here.

Redis follows live trees, not generations

At h31, 2 mature generations and 600 trees hold 4,375 MiB. At h41, 3 generations and 600 trees again hold 3,826 MiB, which is lower, because the third was just born and carries nothing.

CHART 9 · THIS RELEASE

Redis memory and live SMST trees



The tree count is a staircase, because each generation is born in a single block. The memory curve is a ramp, because it follows the relays inserted into those trees.

RELEASES AT EACH CLAIM

Heights	Redis before → after	Released	Live trees
31 → 33	4,375 → 2,461 MiB	-1,914 MiB	600 → 300
51 → 53	5,039 → 2,513 MiB	-2,526 MiB	601 → 301
71 → 73	3,849 → 990 MiB	-2,859 MiB	601 → 300

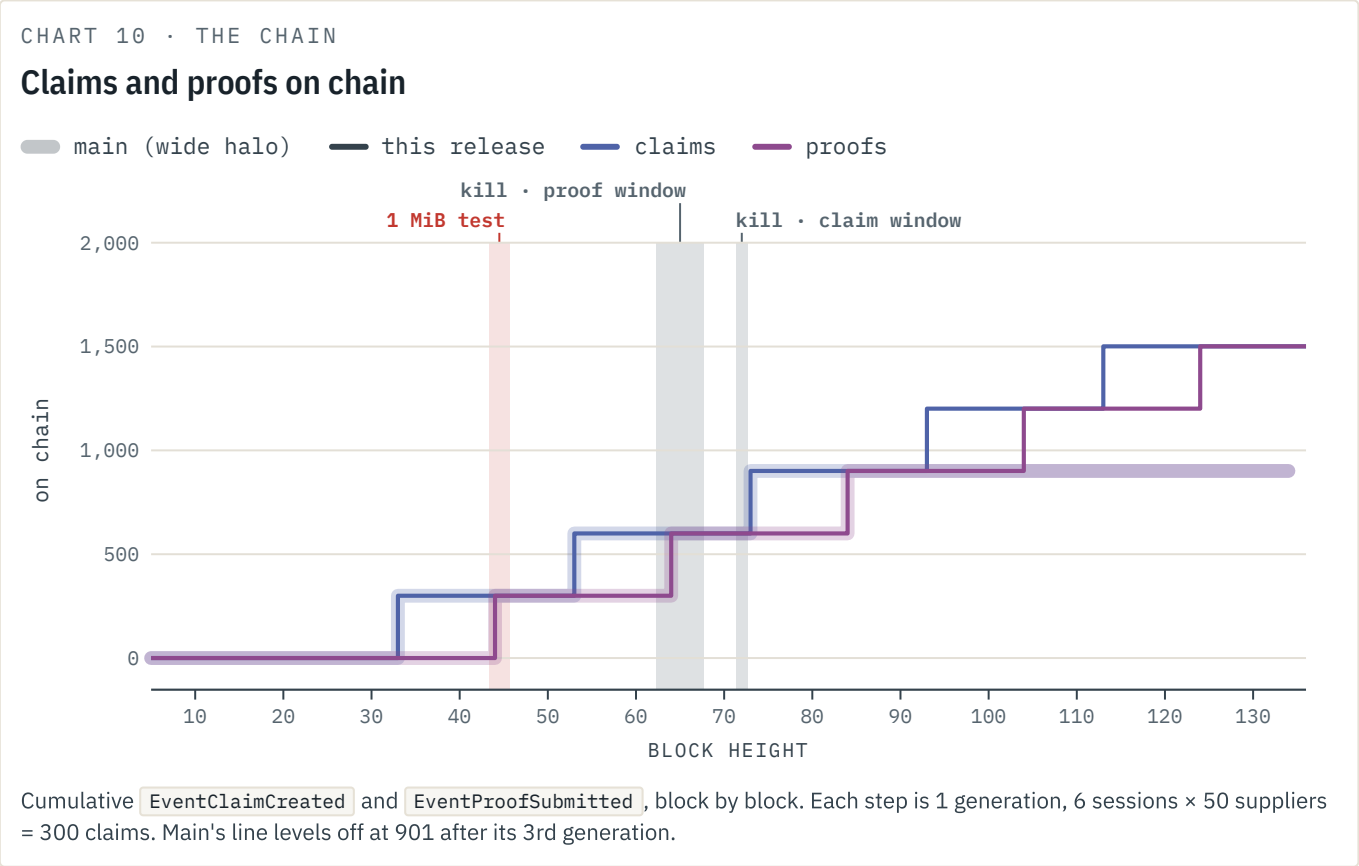
After the first settlement at h53 the chain raised the relay-mining difficulty, which applies to sessions born from h61. From there ~70 % of the relays served no longer enter a tree (19.7 % over the whole run): the protocol stores only relays whose hash meets the difficulty target, and scales the reward to match. That is why Redis grows more slowly afterwards.

The request body is stored whole in its leaf until the proof, while the response is kept only as a hash. Redis therefore grows with the size of the requests, not only with their count; the per-service request size limit is the operator's lever.

Every claim settled

Every claim this release made was proved and settled on chain, including the 2 generations hit by the kills. It counts claims, not relays: whether each claim carried every relay the miner received is not cross-checked here.

Every figure here comes from the chain's block results, not the miner's counters, which reset with each restart.



EACH GENERATION ON CHAIN · CLAIMS · PROOFS · SETTLED (HEIGHT)

Sessions ending at	main	this release
20	300 · 300 · 300	300 h33 · 300 h44 · 300 h53
40 (this release: proof-window kill)	300 · 300 · 300	300 h53 · 300 h64 · 300 h73
60 (this release: claim-window kill)	301 · 301 · 301	301 h73 · 301 h84 · 301 h93
80	no load	300 h93 · 300 h104 · 300 h113
100	no load	300 h113 · 300 h124 · 300 h133

1,501 of 1,501 claims settled, every proof validated, none expired, none slashed. The validator runs its own, unmodified SMST library (v0.14.1), so the chain accepted the new encoding's roots and proofs as they are.

And the relays. Across 10,743,008 relays, 5 were refused because Redis could not record their charge against the application's stake, all at h20–21 (17:45:26Z to 17:45:50Z), well before either test. Every other refusal was a client going away (101, the burst client stopping after its 15 s), a failed relay validation (5) or an unreadable body (1). For each of the 7 services that received relays (6 under load plus a CometBFT probe), relays served equal relays handed to the miner plus relays skipped by difficulty, with a difference of 0.

Setup detail

Why Redis has 2 numbers. `maxmemory` 12.8 GiB with `noeviction`, below the 16 GiB container, is deliberate. With a budget Redis refuses a write instead of being killed by the kernel, and `noeviction` means it never answers pressure by deleting SMST nodes, which would silently throw away billable work. The relayer sees that its charge cannot be recorded and stops admitting until memory frees up. The 3.2 GiB above the budget is Redis's own working room for forks and client buffers.

The load. 1 indexed Kubernetes Job with 6 pods, 1 per service (`develop-http`, `develop-http-eager`, `develop-websocket`, `develop-websocket-optimistic`, `develop-grpc`, `develop-grpc-optimistic`), each re-running a 1,000,000-request load test in a loop at concurrency 8 (1.75–1.83 M relays per service). 30 applications were staked with 10M POKT each; 6 carried load, so stake never ran out. The response check covers HTTP status, the supplier's signature and the JSON-RPC error field.

```
pocket-relay-miner relay jsonrpc \  
  --localnet --service develop-http \  
  --load-test -n 1000000 --concurrency 8 --all-suppliers
```

The other pods differ only in the subcommand (`websocket`, `grpc`) and `--service`.

Source. This release: 2026-09-25 17:29Z to 19:40Z, miner and relayer at 3e99948, built with smt#56 (847bde7) substituted for the released smt library, 1 image for the whole fleet. Main: 2026-09-22 04:01Z to 06:18Z, 0d6ad24 with load-client fixes only and no relayer or miner change.

Evidence. Every figure is read from a run's own artefacts: the per-height sampling (the highest sample of a height for memory and CPU, the mean for throughput and Redis operations), the chain's block results for every height, the per-service identity check, and the logs of the burst and of both kills. Main's grpc and websocket rates and its Redis operations were read from Prometheus at the sampling instants.